

VIÑA CONCHA Y TORO

— FAMILY OF NEW WORLD WINERIES —

MANUAL DESARROLLADOR DSS VCT

Diciembre 2024

Tabla de contenido

1	Introducción.....	6
1.1	Objetivo del proyecto.....	6
1.2	Propuesta del proyecto	6
2	Front-end.....	7
2.1	Instalación	7
2.2	Configuración del Frontend	8
2.2.1	Configuración para Backend Local.....	8
2.2.2	Configuración para Backend en la Nube.....	8
2.3	Levantamiento del Frontend	9
2.4	Actualización del Frontend.....	9
3	Instalación y Levantamiento del Backend.....	9
3.1	Instalación del Backend	9
3.2	Levantamiento (Local)	10
3.1	Actualización del Backend	11
4	Configuración del Solver para Levantamientos del Backend	12
4.1	Levantamiento en Docker	12
4.2	Levantamiento con el Comando Run Server	12
4.3	Levantamiento con Docker (Local)	13
5	Códigos	13
5.1	Lectura de archivos de entrada	14
5.1.1	Consideraciones.....	15
5.1.2	Lectura programación vendimia.....	15
5.1.3	Lectura de API de Planificación de Camiones.....	16
5.1.4	Uso y Transformación de Datos	17
5.1.5	Procesamiento de Datos:.....	18
5.1.6	Cruce de información.....	19
5.1.7	Cruce de Datos:.....	19
5.1.8	Input del Modelo.....	20
5.1.9	Flujo del Proceso	20
5.2	Modelo matemático	21
5.2.1	Funciones.....	21

6	Actualizaciones	37
6.1	Definir usuario: Configuración de Variables	37
6.2	Añadir cambios.....	37
6.2.1	Actualización del Backend.....	37
6.2.2	Actualización del Frontend	38

Índice de figuras

Figura 1: Formato de datos API	15
Figura 2: Librerías necesarias para lectura	15
Figura 3: Función lectura archivo Excel	16
Figura 4: Función lectura API	17
Figura 5: Función obtención de datos	18
Figura 6: Conexión entre archivos de entrada	19
Figura 7: Función ajuste de intervalos	20
Figura 8: Datos de entrada para modelo matemático	20
Figura 9: Lectura maquinarias bodega Lontué	22
Figura 10: Filtrado de datos según hora actual	23
Figura 12: Filtrado de tareas que comenzaron en la ejecución anterior	25
Figura 13: Filtrado de lotes disponibles	26
Figura 14: Definición de parámetro R y Pt	28
Figura 15: Caracterización de tareas	29
Figura 16: Caracterización de estados	30
Figura 17: Definición de variables	31
Figura 18: Función objetivo	31
Figura 19: Función solucionador modelo matemático	32
Figura 20: Función organización de datos	33
Figura 21: Función distribución de kilos	33
Figura 22: Cantidad de uva por camión	34
Figura 23: Función para generar DataFrame con la planificación	34
Figura 24: Resultados ajustados	35
Figura 25: Función de transformación de unidades de medida	35
Figura 26: Configuración de Variables para Cloud Foundry	37
Figura 27: Compilación de Imagen Docker - Backend	37
Figura 28: Subida de imagen a Docker	37
Figura 29: Despliegue en Cloud Foundry - Backend	38
Figura 30: Compilación del Frontend	38

Índice de tablas

Tabla 1: Ejemplo programa vendimia.....	14
Tabla 2: Ejemplo resultados.....	21

1 Introducción

Este manual está diseñado para guiar a los usuarios en el uso y comprensión del Sistema de Apoyo a las Decisiones (DSS, por sus siglas en inglés, Decision Support System) desarrollado para la planificación de la producción de vino. Este innovador software está específicamente diseñado para su implementación en la Planta Lontué de Viña Concha y Toro S.A. El DSS tiene como objetivo mejorar la eficiencia y precisión en la toma de decisiones durante el proceso de producción, integrando datos actualizados y permitiendo una adaptación rápida a los cambios operativos.

Actualmente, el software se encuentra en una etapa inicial de desarrollo, lo que significa que el proyecto está en una fase flexible y susceptible a cambios. Esta fase permite realizar ajustes y mejoras basadas en las pruebas y retroalimentación de los usuarios, asegurando que el sistema final cumpla con las necesidades específicas de la planta y optimice el proceso productivo de manera efectiva.

1.1 Objetivo del proyecto

El objetivo es desarrollar un sistema de soporte para la toma de decisiones, basado en un modelo de optimización, diseñado específicamente para mejorar la planificación diaria en el proceso de producción de vinos blancos en la bodega Lontué de Viña Concha y Toro S.A. Este sistema busca integrar datos actualizados y proporcionar una herramienta flexible y adaptable que permita a los responsables de la producción tomar decisiones más informadas y eficientes. Al optimizar la asignación de recursos y mejorar la coordinación de las actividades, se espera aumentar la eficiencia operativa y la calidad del producto final, contribuyendo así al mantenimiento del liderazgo de Viña Concha y Toro en la industria vitivinícola.

1.2 Propuesta del proyecto

El propósito central del sistema es optimizar la planificación anticipada de las operaciones en la bodega, lo cual es fundamental para mejorar la toma de decisiones y lograr una gestión más eficiente y efectiva.

El sistema propuesto se implementará a través de un entorno web intuitivo y accesible, diseñado para facilitar su uso durante la vendimia. Esta plataforma permitirá la generación

de reportes diarios que sean fáciles de visualizar e interpretar. Estos reportes considerarán factores críticos como las capacidades operativas de las maquinarias, la organización de los grupos de mezcla y la llegada de uvas desde los campos hasta la bodega.

Además, el sistema incluirá herramientas para el monitoreo en tiempo real y la capacidad de ajustar las operaciones según las necesidades cambiantes del proceso productivo. Este enfoque permitirá una mayor flexibilidad y adaptabilidad, asegurando que las decisiones se basen en datos actualizados y precisos.

Este sistema se presenta como un primer paso hacia el desarrollo de una plataforma oficial para la compañía, que en el futuro permitirá la creación de planificaciones con un nivel de detalle y precisión mayor. Con el tiempo, se espera que esta plataforma evolucione para incluir funcionalidades adicionales que mejoren aún más la eficiencia y la calidad del proceso de producción de vino en Viña Concha y Toro.

2 Front-end

2.1 Instalación

Para instalar el Frontend, siga estos pasos:

1. Descargue e instale NodeJS desde nodejs.org/es/download/.
2. Ejecute el instalador correspondiente a su sistema operativo. Asegúrese de incluir Node en el entorno del sistema 'Path' durante la instalación.
3. Abra Visual Studio Code y navegue hasta la carpeta que contiene el código fuente del Frontend.
4. En la terminal de Visual Studio Code, ejecute el siguiente comando para instalar las dependencias del proyecto de React:

```
npm install react-scripts
```

Alternativamente, puede instalar todas las dependencias necesarias con:

```
npm install
```

Si encuentra errores durante la instalación de dependencias, ejecute el siguiente comando para forzar la instalación de la dependencia específica:

```
npm install xxxx --force
```

Este proceso asegurará que todas las herramientas y bibliotecas necesarias estén correctamente configuradas para el desarrollo del Frontend.

2.2 Configuración del Frontend

Para garantizar una comunicación efectiva entre el Frontend y el Backend, es esencial configurar correctamente la dirección del Backend. Esta configuración se lleva a cabo mediante variables de entorno, que se definen en el archivo `.env` ubicado en la raíz del directorio del Frontend.

Las variables de entorno permiten que el Frontend se conecte al Backend de manera dinámica, facilitando la adaptación a diferentes entornos de desarrollo, prueba y producción. Asegúrese de especificar la URL del Backend en el archivo `.env` para que el Frontend pueda interactuar correctamente con los servicios del Backend.

2.2.1 Configuración para Backend Local

Si se opta por una implementación local de todo el sistema, es necesario modificar el archivo `.env` para apuntar al Backend local. Para ello, realice los siguientes ajustes: descomente la línea correspondiente a la dirección local y comente la dirección de Backend preexistente.

```
REACT_APP_BACKEND_URL=http://localhost:8000
```

Este cambio asegurará que el Frontend se comuniquen correctamente con el Backend local durante el desarrollo y las pruebas.

2.2.2 Configuración para Backend en la Nube

En el caso de utilizar un Backend desplegado en la nube, es necesario especificar la URL correspondiente a este entorno en el archivo `.env`. Para ello, realice los siguientes ajustes:

```
REACT_APP_BACKEND_URL=https://capstonebackend.cfapps.br10.hana.ondemand.com
```

Este ajuste asegurará que el Frontend se comuniquen correctamente con el Backend en la nube, permitiendo una integración fluida y eficiente entre ambos componentes del sistema.

2.3 Levantamiento del Frontend

Para iniciar localmente el Frontend desarrollado con React, siga estos pasos:

1. Abra la consola o terminal integrada en Visual Studio Code y navegue hasta el directorio específico denominado 'Frontend'.
2. En la terminal, ejecute el siguiente comando para iniciar la aplicación:

```
npm start
```

3. Si encuentra errores relacionados con dependencias faltantes, instale la dependencia específica que falta mediante el siguiente comando:

```
npm install xxxx
```

Es importante recordar que estos pasos están diseñados para la ejecución local del Frontend. En entornos de despliegue como Cloud Foundry, la inicialización del Frontend puede gestionarse de manera automática al actualizar el código fuente.

2.4 Actualización del Frontend

Cuando se realizan cambios en el código del Frontend, los pasos a seguir a modo general son los siguientes:

1. Compilar el Frontend:
 - Desde PowerShell en Visual Studio, navegue a la carpeta del Frontend.
 - Compile el proyecto antes de subirlo a Cloud Foundry:
2. Subir cambios a Cloud Foundry
 - Abra un **Command Prompt** en Visual Studio, ubicado en la carpeta del Frontend.
 - Arrastre el archivo **push-dev.cmd** desde la carpeta del proyecto a la terminal y ejecute el comando.

3 Instalación y Levantamiento del Backend

3.1 Instalación del Backend

Para configurar el backend del proyecto, sigue los pasos detallados a continuación:

1. **Instalación de Python:** Descarga e instala la última versión de Python desde su sitio oficial: <https://www.python.org/downloads/>. Este paso también habilitará el uso del administrador de paquetes pip.
2. **Instalación de Virtualenv:** Ejecuta el siguiente comando en la terminal para instalar virtualenv:

```
pip install virtualenv
```

3. **Creación y activación de un entorno virtual:** Una vez instalado virtualenv, crea y activa un entorno virtual con los siguientes comandos:

```
virtualenv venv  
.\venv\Scripts\activate
```

4. **Instalación de dependencias:** Con el entorno virtual activado, instala las dependencias del proyecto utilizando el archivo requirements.txt:

```
pip install -r requirements.txt
```

5. **Verificación de dependencias:** Para asegurarte de que las dependencias se instalaron correctamente, ejecuta el siguiente comando para iniciar el servidor local:

```
python manage.py runserver
```

Si todo está en orden, la API debería funcionar correctamente. En caso de errores relacionados con dependencias faltantes, instala manualmente la dependencia específica con:

```
pip install nombre_de_la_dependencia
```

3.2 Levantamiento (Local)

Sigue estos pasos para levantar el backend en un entorno local:

1. **Activación del entorno virtual:** Activa el entorno virtual con el comando:

```
.\venv\Scripts\activate
```

2. **Ejecución de migraciones (si es necesario):** Realiza las migraciones de la base de datos con los siguientes comandos:

```
python manage.py makemigrations dssProject python manage.py migrate dssProject
```

3. **Levantamiento del servidor:** Inicia la API ejecutando:

```
python manage.py runserver
```

El backend debería estar ahora en funcionamiento. Estas instrucciones aplican exclusivamente para un entorno local. En entornos como Cloud Foundry, el levantamiento puede ser gestionado automáticamente al actualizar el código (ver sección 6.2.1).

3.1 Actualización del Backend

Cuando se realizan cambios en el código del Backend, los pasos a seguir a modo general son los siguientes:

1. Definir variables de usuario y contraseña (Cloud Foundry):
 - Abra una terminal CMD con permisos de administrador.
 - Configure las credenciales necesarias para la conexión con Cloud Foundry
2. Compilar la imagen de Docker
 - Desde PowerShell en Visual Studio, ubicado en la raíz del proyecto, compile la imagen del Backend.
 - Asegúrese de que Docker esté ejecutándose.
3. Subir la imagen a Docker Hub
 - Abra una terminal CMD con permisos de administrador.
 - Inicie sesión en Docker Hub y suba la imagen
4. Subir cambios a Cloud Foundry
 - Abra un **Command Prompt** en Visual Studio, ubicado en la carpeta del Frontend.
 - Arrastre el archivo **push-dev.cmd** desde la carpeta del proyecto a la terminal y ejecute el comando.

Abra una terminal CMD con permisos de administrador.

Configure las credenciales necesarias para la conexión con Cloud Foundry:

4 Configuración del Solver para Levantamientos del Backend

Al configurar el backend en Docker o utilizar el comando runserver, es importante definir correctamente el solver que se utilizará. A continuación, se detallan las instrucciones según el entorno de ejecución:

4.1 Levantamiento en Docker

En el archivo optback.py, ubica la línea y asegúrate de que el solver esté configurado de la siguiente manera:

```
% En el archivo optback.py
% Configuración del solver para ejecución en Docker
solver = SolverFactory('cbc')
```

Verifica que tu imagen de Docker incluya el solver necesario y todas las dependencias requeridas.

4.2 Levantamiento con el Comando Run Server

En el archivo optback.py, ubica la línea y descomenta las siguientes líneas y comenta la configuración para Docker:

```
% En el archivo optback.py
% Configuración del solver para ejecución con el comando run server
% Descomente las siguientes líneas y comente la configuración para Docker
% current_dir = os.path.dirname(__file__)
% parent_dir = os.path.join(os.path.dirname(current_dir), 'Pyomo', 'CBC', 'bin', 'cbc')
% solver = SolverFactory('cbc', executable=parent_dir)
```

Asegúrate de que la ruta del ejecutable del solver sea correcta para tu entorno local, según la estructura específica de su proyecto y su entorno de ejecución.

4.3 Levantamiento con Docker (Local)

Para levantar el Backend utilizando Docker de manera local:

1. **Archivo Dockerfile:** Asegúrate de que el directorio del proyecto contenga un archivo Dockerfile con la configuración necesaria.

2. **Modificación del Dockerfile:**

- Comenta la siguiente línea si estás ejecutando localmente:

```
#Comente la siguiente línea si está ejecutando localmente  
#ENTRYPOINT ["gunicorn", "backAPI.wsgi", "--timeout", "3600", "--bind", "0.0.0.0:80"]
```

- Descomenta esta línea para ejecución local:

```
#Descomente la siguiente línea si está ejecutando localmente  
#CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

3. **Construcción de la imagen Docker:** Construye la imagen Docker con el comando:

```
docker build -t nombre-de-la-imagen .
```

4. **Ejecución del contenedor Docker:** Ejecuta el contenedor con:

```
docker run -p 8000:8000 nombre-de-la-imagen
```

5. **Acceso a la aplicación:** La aplicación Django estará disponible en <http://localhost:8000/>.

Recuerda que estas instrucciones son específicas para ejecución local. En entornos como Cloud Foundry, la gestión del levantamiento puede variar y realizarse automáticamente al actualizar el código.

5 Códigos

En esta sección se detalla la funcionalidad de los códigos utilizados en el sistema, incluyendo explicaciones sobre cómo funcionan y posibles configuraciones que se pueden

realizar. Esto abarca desde la lectura de archivos de entrada hasta el procesamiento de datos para alimentar el modelo matemático.

5.1 Lectura de archivos de entrada

Para que el sistema funcione correctamente, dos inputs son necesarios:

1. **Archivo de programación de vendimia:** Este archivo es generado por el área enológica de la bodega Lontué en formato Excel. Contiene información sobre los productores, fundos, y bloques de mezcla asociados al tipo de vino que se producirá. A continuación, se muestra un ejemplo del formato en la Tabla 1:

Tabla 1: Ejemplo programa vendimia

Fuente: Área enología bodega Lontué.

CTTO	PRODUCTOR	VARIEDAD	CODIGO	BLOQUE	KG CTTO	PROGRAMADO
(integer)	(string)	(string)	(integer)	(integer)	(integer)	(integer)

2. API de planificación de camiones

El modelo se conecta a una API del software de planificación de camiones que proporciona datos en tiempo real sobre la logística de transporte. La API devuelve información clave para la planificación, incluyendo el horario de llegada estimado de los camiones a la bodega, productor, código de variedad, entre otros.

3. Formato de Datos Recibidos desde la API

La API retorna un conjunto de datos estructurados en formato JSON. A continuación, se describe la estructura del JSON recibido:

```

{
  "viajes": [
    {
      "Hora de llegada a bodega estimada":
      "Productor":
      "Bodega":
      "Codigo Material":
      "Patente":
      "Kilos programados":
      "Nombre del materia":
      "Fundo origen":
      "Estado\u00a0del\u00a0viaje":
    }
  ]
}

```

Figura 1: Formato de datos API

Fuente: Elaboración propia.

Al momento de hacer la lectura de ambos archivos, se debe realizar un cruce de información mediante la variedad y el número de contrato para obtener el bloque de mezcla que pertenece el pedido de la planificación diaria.

5.1.1 Consideraciones

Para la lectura de ambos las librerías utilizadas en python son las siguientes:

```

import requests
import json
import datetime as dt
import math
import datetime
import pytz

```

Figura 2: Librerías necesarias para lectura

Fuente: Elaboración propia.

5.1.2 Lectura programación vendimia

En esta sección se describe el código actualizado para la lectura y limpieza del archivo "Programa Vendimia" en formato Excel. Este archivo contiene información clave sobre los contratos, productores, variedades y bloques de mezcla necesarios para la planificación de la vendimia.

La función **_load_excel_data** es un componente auxiliar que realiza la lectura inicial del archivo Excel, omitiendo las primeras cuatro filas que contienen información innecesaria.

La función **cleaning_vendimia** se encarga de procesar y estructurar los datos para su uso en el modelo matemático, garantizando que solo se incluyan datos válidos y relevantes, según se necesiten.

```
def _load_excel_data(file_path):
    """Carga el archivo Excel."""
    return pd.read_excel(file_path, skiprows=4)

def cleaning_vendimia(file_path):
    """Limpia los datos del programa de vendimia."""
    df = _load_excel_data(file_path)
    columnas_deseadas = ['CTTO', 'PRODUCTOR', 'VARIEDAD', 'CODIGO', 'BLOQUE', 'PROGRAMADO']
    df = df[columnas_deseadas]
    #df['CTTO'] = pd.to_numeric(df['CTTO'], errors='coerce').astype(int)
    df['CTTO'] = df['CTTO'].astype(str)
    df['BLOQUE'] = df['BLOQUE'].fillna(0).astype(int)
    df['CODIGO'] = df['CODIGO'].fillna(0).astype(str)
    df['PROGRAMADO'] = df['PROGRAMADO'].fillna(0).astype(int)
    df = df[df['PROGRAMADO'] != 0]
    return df
```

Figura 3: Función lectura archivo Excel.

Fuente: Elaboración propia.

5.1.3 Lectura de API de Planificación de Camiones

En esta sección se describe cómo se realiza la consulta y extracción de datos de la API de planificación de camiones.

```

def fetch_data_from_api(current_date):
    url = f"https://apiplan.divan.tech:8443/divanplan/viajes2?fecha={current_date}&bodega=BODEGA LONTUE BLEND"
    user = "consulta"
    password = "1234"

    try:
        response = requests.get(url, auth=(user, password))
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)

        # Process the successful response
        data = response.json() # Assuming the API returns JSON data
        formatted_json = json.dumps(data, indent=4)
        print(formatted_json)

        # Extract the desired data and create a DataFrame
        extracted_data = []
        for item in data['viajes']:
            extracted_data.append({
                'Hr Bodega': item['Hora de llegada a bodega estimada'],
                'Productor': item['Productor'],
                'Fundo': item['Fundo origen'],
                'ID_Material': item['Codigo Material'],
                'Patente': item.get('Patente', 'Sin Patente'),
                'Kilos': item['Kilos programados'],
                'Estado': item['Estado\u00a0del\u00a0viaje'],
                'Fecha': current_date,
                'Contrato': item.get('CTTO', 'Sin Contrato'),
                'Num. Viaje': item.get('Num. Viaje', 'Sin Num. Viaje')
            })

        return pd.DataFrame(extracted_data)

    except requests.exceptions.RequestException as e:
        print(f"An error occurred: {e}")
        return pd.DataFrame() # Return an empty DataFrame on error
    except json.JSONDecodeError as e:
        print(f"Invalid JSON response: {e}")
        return pd.DataFrame() # Return an empty DataFrame on error

```

Figura 4: Función lectura API

Fuente: Elaboración propia.

La función **fetch_data_from_api** realiza solicitudes HTTP a la API con credenciales básicas de autenticación. Extrae la información observada el punto 5.1 y modifica los nombres de las columnas del DataFrame al que son agregados los datos. La url funciona en base a un *current_date* que se puede observar en el siguiente punto.

5.1.4 Uso y Transformación de Datos

El siguiente bloque utiliza la función **fetch_data_from_api** para obtener los datos de dos fechas consecutivas y realizar transformaciones específicas:

```

# Get current date in YYYY-MM-DD format
current_date = datetime.date.today().strftime('%Y-%m-%d')
current_date = '2024-03-20' #Comentar para usar Hora
current_date_dt = dt.datetime.strptime(current_date, '%Y-%m-%d').date()

# Calculate next_date by adding timedelta
next_date = (current_date_dt + datetime.timedelta(days=1)).strftime('%Y-%m-%d')

# Fetch data for current and next date
df_c = fetch_data_from_api(current_date)
df_n = fetch_data_from_api(next_date)

# Create 'Hr Bodega Num' column
if not df_c.empty:
    df_c['Hr Bodega Num'] = pd.to_datetime(df_c['Hr Bodega']).dt.hour + pd.to_datetime(df_c['Hr Bodega']).dt.minute / 60
else:
    print("df_c is empty, skipping 'Hr Bodega Num' transformation.")

if not df_n.empty:
    df_n['Hr Bodega Num'] = pd.to_datetime(df_n['Hr Bodega']).dt.hour + pd.to_datetime(df_n['Hr Bodega']).dt.minute / 60
else:
    print("df_n is empty, skipping 'Hr Bodega Num' transformation.")

# Apply filters with a check for empty DataFrames
if not df_c.empty:
    df_c = df_c[(df_c['Hr Bodega Num'] > 6) | (df_c['Estado'] != 'c')]
#df_c = df_c[df_c['Hr Bodega Num'] > 6]
else:
    print("df_c is empty, skipping filter for > 6 o estado de viaje != c")

if not df_n.empty:
    df_n = df_n[df_n['Hr Bodega Num'] <= 6]
else:
    print("df_n is empty, skipping filter for <= 6.")

# Combine the filtered DataFrames with a check for empty df_n
if not df_n.empty:
    all_data = pd.concat([df_c, df_n])
else:
    all_data = df_c # Assign df_c to all_data if df_n is empty
    print("df_n is empty, all_data will only contain data from df_c.")

all_data.reset_index(drop=True, inplace=True)

```

Figura 5: Función obtención de datos

Fuente: Elaboración propia.

5.1.5 Procesamiento de Datos:

Se convierten las horas de llegada (Hr Bodega) a un formato numérico decimal (Hr Bodega Num).

Se filtran los registros:

- **Día actual:** Horas posteriores a las 06:00 AM o con estado diferente de "C".
- **Día siguiente:** Horas iguales o anteriores a las 06:00 AM.

Current_Date tiene 2 opciones, la primera es la que deberá ser utilizada en Vendimia, mientras que la segunda, puede ser utilizada para pruebas de un día específico:

- `current_date = datetime.date.today().strftime('%Y-%m-%d')`
- `current_date = '2024-03-20'`

5.1.6 Cruce de información

En la función **merge_api_and_vendimia** se realiza la conexión entre ambos inputs de entrada, donde básicamente se mezcla la información el dataframe obtenido de la API con el dataframe de vendimia, obteniendo así el “Bloque” respectivo a las llegadas de la API.

```
def merge_api_and_vendimia(dfAPI, df_vendimia):
    # Merge the DataFrames based on 'CODIGO' and 'ID_Material'
    df_final = pd.merge(df_vendimia, dfAPI, left_on='CODIGO', right_on='ID_Material', how='inner')

    # Select the desired columns
    df_final = df_final[['Hr Bodega', 'Kilos', 'BLOQUE', 'Patente', 'Fecha', 'Hr Bodega Num', 'Estado']]

    # Rename the 'BLOQUE' column
    df_final = df_final.rename(columns={'BLOQUE': 'Bloque'})

    # Remove duplicate rows
    df_final = df_final.drop_duplicates()

    # Filter out rows where 'Bloque' is equal to 0
    df_final = df_final[df_final['Bloque'] != 0]

    # Remove duplicates based on specific columns
    df_final = df_final.drop_duplicates(subset=['Hr Bodega', 'Kilos', 'Patente'])

    # Reset index
    df_final.reset_index(drop=True, inplace=True)

    return df_final
```

Figura 6: Conexión entre archivos de entrada

Fuente: Elaboración propia.

5.1.7 Cruce de Datos:

- Se utiliza un merge entre los DataFrames `df_vendimia` (programa de vendimia) y `dfAPI` (datos de la API) utilizando las columnas `CODIGO` e `ID_Material` como claves.
- Se eliminan aquellas filas donde el bloque es igual a 0, esto ya que en el cruce se trae información del dataframe de vendimia que podían estar vacías.

5.1.8 Input del Modelo

Como ajustes finales, la función `_discretize_hour` se utiliza para corregir la lógica de intervalos, es decir, redondea hacia arriba si los minutos son mayores o iguales a 30, caso contrario, redondea hacia abajo.

```
def _discretize_hour(hour):  
    return np.ceil(hour) if (hour % 1 >= 0.5) else np.floor(hour)
```

Figura 7: Función ajuste de intervalos

Fuente: Elaboración propia.

En la función `process_vendimia_data` se lee el dataframe con la conexión realizada simplificándolo con el formato para la entrada del modelo, donde específicamente se quiere obtener la hora en decimal, bloque de mezcla y la cantidad de kilos recibidos.

```
def process_vendimia_data(input_df):  
    """Procesa el DataFrame de vendimia para agregar columnas discretizadas."""  
    input_df['Hr Bodega Up'] = input_df['Hr Bodega Num'].apply(_discretize_hour)  
  
    # Define intervals and labels for discretization  
    intervalos = [[i, i + 1] for i in range(0, 30, 1)]  
    miniTIME = [i for i in range(len(intervalos))]  
  
    # Create the 't' column based on the discretized 'Hr Bodega Up'  
    input_df['t'] = pd.cut(input_df['Hr Bodega Up'],  
                           bins=[intervalo[0] for intervalo in intervalos] + [intervalos[-1][1]],  
                           labels=miniTIME, right=False)  
  
    # Sort the DataFrame by 'Fecha', 'Hr Bodega', and 't'  
    input_df.sort_values(by=['Fecha', 'Hr Bodega', 't'], inplace=True)  
  
    return input_df
```

Figura 8: Datos de entrada para modelo matemático

Fuente: Elaboración propia.

5.1.9 Flujo del Proceso

1. Discretización de la Hora:

- Se aplica una función personalizada (`_discretize_hour`) que redondea la hora en formato decimal:
 - Redondea hacia arriba si los minutos son mayores o iguales a 30.
 - Redondea hacia abajo si los minutos son menores a 30.

- El resultado se almacena en la columna Hr Bodega Up.

2. Definición de Intervalos de Tiempo (t):

- Se crean intervalos de tiempo (e.g., 0-1, 1-2, ..., 29-30) y se asignan etiquetas numéricas (miniTIME) para identificar los periodos.

3. Creación de la Columna t:

- Se utiliza pd.cut para asignar cada registro a un intervalo según la hora discretizada (Hr Bodega Up).

4. Ordenación de Datos:

- Los registros se ordenan por Fecha, Hr Bodega (hora original) y t (intervalo de tiempo).

5. Resultado:

- El DataFrame resultante (df_final_processed) contiene las columnas necesarias para la entrada al modelo matemático.

Tabla 2: Ejemplo resultados

Fuente: Elaboración propia.

Fecha	Hr Bodega	Hr Bodega Num	Hr Bodega Up	t	Kilos	Bloque	Patente
2024-03-20	07:30:00	7.5	8	8	10000	12	AB123CD
2024-03-20	09:45:00	9.75	10	10	8000	8	XY456ZT
2024-03-20	06:15:00	6.25	6	6	9500	5	GH789IJ

5.2 Modelo matemático

5.2.1 Funciones

- **preprocess_machines_data:** Esta función tiene como objetivo realizar un preprocesamiento de las máquinas disponibles en la bodega, recopilando información como sus capacidades, tiempos de procesamiento, y nombres.

Adicionalmente, incluye un análisis de los requerimientos de las cubas de fermentación, ajustando la cantidad utilizada según la cantidad de uva que se recibe durante el día. De las 230 cubas disponibles por defecto, solo se utilizan las necesarias, lo que contribuye a reducir los tiempos de espera.

Es importante destacar que la capacidad mínima de las máquinas se establece inicialmente en un 40 % de su capacidad máxima, con la posibilidad de ajustarla según las preferencias del usuario.

Por último, se aplica una tasa de conversión en los procesos posteriores al prensado para unificar las asignaciones internas del modelo en una única unidad de medida: toneladas. Esto simplifica los cálculos y asegura coherencia en las asignaciones.

```
def preprocess_machines_data(file_path):
    df = pd.read_excel(file_path)
    df['Bmin'] = df['Bmax']*0.4
    CubasDisp = df[df['Máquina'].str.startswith('CubaF')]
    df=df[df['Estado'] == 'Habilitado'].reset_index(drop=True)
    Bminprom = CubasDisp['Bmin'].mean()
    ReqCfer = round((ReqTon*0.96*0.84*0.94*760/Bminprom),0)
    df['Estado'] = ['Deshabilitado' if 'CubaF' in maquina and int(maquina.split('_')[1]) > ReqCfer else 'Habilitado' for maquina in df['Máquina']]
    df=df[df['Estado'] == 'Habilitado'].reset_index(drop=True)
    rho_convert = {'Despalillado': 1.0,
                   'Prensado': 1.0,
                   'Pre-flotación': 1/760,
                   'Flotación': 1/760,
                   'Fermentación': 1/760}
    UNITS_TASKS = {(df['Máquina'][i],df['Tarea'][i]): {'Bmin': round(df['Bmin'][i]*rho_convert[df['Tarea'][i]],0),
                                                         'Bmax': round(df['Bmax'][i]*rho_convert[df['Tarea'][i]],0),
                                                         'T. Proc': df['T. Proc'][i], 'Tclean':0} for i in range(len(df))}

    return UNITS_TASKS
```

Figura 9: Lectura maquinarias bodega Lontué

Fuente: Elaboración propia.

- **obtener_hora_actual:** Esta función obtiene la hora actual en la zona horaria de Santiago de Chile y devuelve únicamente la hora (en formato de 24 horas). Utiliza la hora UTC y la convierte a la hora local de Santiago.
- **definir_hora_horizonte:** Esta función define la hora de planificación y el horizonte de planificación en función de la hora actual. Establece tres bloques horarios (9, 13 y 20) para determinar el inicio y el fin del horizonte de planificación, ajustándose dinámicamente según la hora proporcionada. Devuelve tanto la hora de planificación como el horizonte correspondiente.

- **ajustar_columna_t:** Esta función ajusta los valores de la columna 't' en un DataFrame. Convierte la columna a valores numéricos, corrige los valores menores a 7 sumándoles 24 y ajusta los valores entre 7 y 9 estableciéndolos en 9. Devuelve el DataFrame modificado.
- **filtrar_input:** Esta función filtra un DataFrame de entrada basado en la hora actual. Define rangos de planificación (H1, H2, H3) y utiliza la hora actual para seleccionar registros donde la columna 't' cumpla ciertas condiciones o donde el 'Estado de viaje' sea distinto de 'c'. También ordena los datos por la columna 't' de manera ascendente. Devuelve el DataFrame filtrado y el rango de horas relevante (H_).

```
def filtrar_input(df, current_hour):
    global H1, H2, H3
    H1 = 9
    H2 = 13
    H3 = 20
    maxtNum = 7
    if current_hour < H1:
        H_ = range(H1, int(maxtNum))
        filtered_df = df[(df['t'] >= (H1)) | (df['Estado'] != 'C')]
    elif H1 <= current_hour < H2:
        H_ = range(H2, int(maxtNum))
        filtered_df = df[(df['t'] >= (H2)) | (df['Estado'] != 'C')]
    else:
        H_ = range(H3, int(maxtNum))
        filtered_df = df[(df['t'] >= (H3)) | (df['Estado'] != 'C')]

    filtered_df.sort_values(by=['t'], ascending=True, inplace=True)
    return filtered_df, H_
```

Figura 10: Filtrado de datos según hora actual

Fuente: Elaboración propia.

- **procesar_input_y_guardar_recibo:** Esta función procesa un DataFrame filtrado para extraer información clave y guardarla en un archivo Excel. Agrupa los datos por hora ('t') y bloque ('Bloque'), calcula la suma de los kilos para cada grupo (convertidos a toneladas), y organiza los resultados en un DataFrame llamado Recieve. Además, identifica los bloques únicos como un conjunto llamado

GROUPS. Finalmente, guarda Recieve en un archivo llamado 'input_model.xlsx' y devuelve tanto GROUPS como Recieve.

- **filtrar_y_agrupar_resultados:** La función comienza filtrando el DataFrame de entrada (df_resultados) para identificar las tareas que son relevantes en el contexto de la hora de planificación actual. En particular, se divide el análisis en dos tipos de datos: las tareas ya finalizadas y las tareas de "Prensado".

Primero, se seleccionan las tareas cuyo tiempo de finalización ('Fin') es menor o igual a "hora_planificacion", ya que estas se consideran como terminadas. A estas tareas se les calcula la suma de los tamaños ('Tamaño [Ton]') y de los lotes de salida ('Lote de salida') agrupándolas por 'Bloque' (bloque de mezcla) y 'Tarea' (etapa del proceso). El resultado se almacena en un DataFrame llamado 'grouped_results'.

Luego, se realiza un tratamiento especial para las tareas categorizadas como "Prensado". Este análisis se centra en las tareas cuyo tiempo de inicio ('Inicio') es anterior a "hora_planificacion", pero cuya finalización ('Fin') ocurre después de este momento, es decir, tareas que están en progreso. Estas tareas se agrupan adicionalmente por 'Fin', para capturar los tiempos específicos en los que concluirán. Nuevamente, se calculan las sumas de 'Tamaño [Ton]' y 'Lote de salida', y el resultado se guarda en un DataFrame llamado 'prensado_results'.

Finalmente, si hay datos válidos en el conjunto de "Prensado", estos se combinan con los resultados ya agrupados en "grouped_results". Esto asegura que tanto las tareas completadas como aquellas en progreso estén representadas en los resultados consolidados.

```

def filtrar_y_agrupar_resultados(df_resultados, hora_planificacion):
    filtered_results = df_resultados[df_resultados['Fin'] <= hora_planificacion]
    grouped_results = filtered_results.groupby(['Bloque', 'Tarea']).agg({
        'Tamaño [Ton]': 'sum',
        'Lote de salida': 'sum'
    }).reset_index()

    prensado_results = df_resultados[df_resultados['Tarea'] == 'Prensado']
    prensado_results = prensado_results[(prensado_results['Fin'] > hora_planificacion) &
                                         (prensado_results['Inicio'] < hora_planificacion)]

    if not prensado_results.empty:
        prensado_results = prensado_results.groupby(['Bloque', 'Tarea', 'Fin']).agg({
            'Tamaño [Ton]': 'sum',
            'Lote de salida': 'sum'
        }).reset_index()
        grouped_results = pd.concat([grouped_results, prensado_results], ignore_index=True)

    return grouped_results, prensado_results

```

Figura 11: Filtrado de tareas que comenzaron en la ejecución anterior

Fuente: Elaboración propia.

- **procesar_resultados_agrupados:** Esta función ajusta los lotes de salida entre tareas consecutivas, asegurando que los recursos se asignen correctamente. Para cada tarea, verifica si la siguiente tarea tiene bloques coincidentes y ajusta los lotes de salida restando el tamaño de los bloques de la tarea siguiente. Si no hay registros para la tarea siguiente, mantiene los valores originales. El resultado es un DataFrame con los lotes de salida actualizados

```

def procesar_resultados_agrupados(grouped_results):

    tareas = grouped_results['Tarea'].unique()
    resultado_final = pd.DataFrame(columns=grouped_results.columns)

    for idx, tarea_actual in enumerate(tareas):
        tarea_actual_data = grouped_results[grouped_results['Tarea'] == tarea_actual]

        if idx < len(tareas) - 1:
            tarea_siguiente = tareas[idx + 1]
            tarea_siguiente_data = grouped_results[grouped_results['Tarea'] == tarea_siguiente]

            for _, row_actual in tarea_actual_data.iterrows():
                bloque = row_actual['Bloque']
                lote_salida = row_actual['Lote de salida']

                registros_siguientes = tarea_siguiente_data[tarea_siguiente_data['Bloque'] == bloque]
                if not registros_siguientes.empty:
                    total_tamano_siguiente = registros_siguientes['Tamaño [Ton]'].sum()
                    disponible = lote_salida - total_tamano_siguiente
                    if disponible > 0:
                        row_actual['Lote de salida'] = disponible
                        if not pd.DataFrame([row_actual]).empty:
                            resultado_final = pd.concat([resultado_final, pd.DataFrame([row_actual])], ignore_index=True)
                    else:
                        if not pd.DataFrame([row_actual]).empty:
                            resultado_final = pd.concat([resultado_final, pd.DataFrame([row_actual])], ignore_index=True)
                else:
                    resultado_final = pd.concat([resultado_final, tarea_actual_data], ignore_index=True)

    return resultado_final

```

Figura 12: Filtrado de lotes disponibles

Fuente: Elaboración propia.

- **create_VCT:** Esta función define las condiciones iniciales de la bodega, estableciendo diversos parámetros clave para la gestión del proceso de producción. Los puntos importantes son:
 1. **TIME:** La función ajusta el horizonte de tiempo en función de la cantidad de valores a programar y la hora actual. Según el número de entradas (cantidad_input), el rango de tiempo (TIME) se configura para adaptarse a diferentes periodos:
 - Si cantidad_input es mayor a 70, se selecciona uno de tres posibles rangos horarios basados en la hora actual (H1, H2, H3). Dependiendo de en qué rango se encuentre la hora actual, se crean intervalos de tiempo cuya planificación finaliza entre las 20:00 horas y las 07:00 horas del día siguiente.
 - Si cantidad_input es mayor a 50 pero menor o igual a 70, la función ajusta el horizonte a las horas de las 03 o 07, según la hora actual

- Si cantidad_input es menor o igual a 50, se asigna un rango completo de 31 horas (7 a.m. día siguiente).
2. STATES: Se definen los estados involucrados en el proceso, especificando su capacidad, cantidad inicial y precio. Estos estados incluyen productos como el racimo de uva, jugo, orujo, entre otros. El precio refleja los costos asociados, lo que permite priorizar ciertos estados, como la uva, para evitar su oxidación.
 3. ST_ARCS: Se detallan los estados de entrada para cada proceso, como el racimo de uva en el despallado, con un 100% de eficiencia en la transición entre los estados.
 4. TS_ARCS: Define los estados generados en cada tarea, con su respectiva proporción de salida, como en el caso del despallado que genera un 96% de uva despallada y un 4% de escobajo, con cantidades ajustables según las necesidades.

Esta función no solo establece los parámetros iniciales de los estados y arcos, sino que también adapta dinámicamente el horizonte de tiempo según la carga de programación y la hora actual, optimizando la planificación del proceso de producción

- **initialize_R_and_Pt:** La función initialize_R_and_Pt tiene como objetivo inicializar dos diccionarios: Pt, que almacena las patentes asociadas a cada grupo y tiempo, y R, que guarda la cantidad de material para un estado, grupo y tiempo específicos. Inicialmente, ambos diccionarios están vacíos o con valores por defecto.

Luego, la función llena estos diccionarios con datos provenientes de tres fuentes. Primero, recorre el dataframe Recieve para asignar los kilos recibidos a un estado y tiempo específico en R, y añade las patentes correspondientes en Pt. Después, actualiza R con los resultados de resultado_final, asociando los lotes de salida a un estado y hora de planificación según la tarea realizada. Finalmente, también actualiza R con los datos de prensado_results, registrando los lotes de salida del proceso de prensado y su tiempo de finalización.

Al final, la función devuelve los diccionarios R y Pt actualizados.

```

def initialize_R_and_Pt(Recieve, filtered_input, resultado_final, prensado_results, STATES, GROUPS, TIME, hora_planificacion):
    # Inicializar Pt y R
    Pt = {(g, t): [] for g in list(GROUPS) for t in TIME} # Convert GROUPS to list
    R = {(s, g, t): 0 for s in STATES for g in list(GROUPS) for t in TIME} # Convert GROUPS to list

    # Llenar R con los datos de Recieve
    for i in range(len(Recieve)):
        R['Racimo_uva', Recieve['Bloque'][i], Recieve['t'][i]] = Recieve['Kilos'][i]

        clave = (Recieve['Bloque'][i], Recieve['t'][i])
        if clave in Pt: # Verificar si la clave existe
            Pt[clave].extend(filtered_input.loc[
                (filtered_input['Bloque'] == Recieve['Bloque'][i]) &
                (filtered_input['t'] == Recieve['t'][i]),
                'Patente'
            ].tolist())

    # Llenar R con los datos de resultado_final
    for index, row in resultado_final.iterrows():
        bloque = row['Bloque']
        tarea = row['Tarea']
        lote_salida = row['Lote de salida']

        # Definir el estado según la etapa
        estado = {
            'Despalillado': 'Uva_despalillada',
            'Pre-flotación': 'Jugo_uva_II',
            'Flotación': 'Mosto',
            'Fermentación': 'Vino'
        }.get(tarea)

        if estado: # Si el estado es válido
            R[estado, bloque, hora_planificacion] = lote_salida
            print(f"estado: {estado}, bloque: {bloque}, hora_planificacion: {hora_planificacion}, lote_salida: {lote_salida}")

    # Llenar R con los datos de prensado_results
    for index, row in prensado_results.iterrows():
        bloque = row['Bloque']
        tarea = row['Tarea']
        lote_salida = row['Lote de salida']
        fin = row['Fin'] # Usar 'Fin' como el tiempo

        # Definir el estado según la etapa
        estado = 'Jugo_uva_I' if tarea == 'Prensado' else None

        if estado: # Si el estado es válido
            R[estado, bloque, fin] = lote_salida
            print(f"estado: {estado}, bloque: {bloque}, fin: {fin}, lote_salida: {lote_salida}")

    return R, Pt

```

Figura 13: Definición de parámetro R y Pt

Fuente: Elaboración propia.

- **characterize_tasks:** Se establecen los diversos conjuntos asociados a los procesos, incluyendo los nombres de las tareas, los estados de entrada y salida junto con sus respectivas tasas. Asimismo, se detallan las máquinas que tienen la capacidad de ejecutar cada proceso específico.

```

def characterize_tasks(UNIT_TASKS, ST_ARCS, TS_ARCS):
    # set of tasks
    TASKS = set([i for (j,i) in UNIT_TASKS])

    # S[i] input set of states which feed task i
    S = {i: set() for i in TASKS}
    for (s,i) in ST_ARCS:
        S[i].add(s)

    # S_[i] output set of states fed by task i
    S_ = {i: set() for i in TASKS}
    for (i,s) in TS_ARCS:
        S_[i].add(s)

    # rho[(i,s)] input fraction of task i from state s
    rho = {(i,s): ST_ARCS[(s,i)]['rho'] for (s,i) in ST_ARCS}

    # rho_[(i,s)] output fraction of task i to state s
    rho_ = {(i,s): TS_ARCS[(i,s)]['rho'] for (i,s) in TS_ARCS}

    # K[i] set of units capable of task i
    K = {i: set() for i in TASKS}
    for (j,i) in UNIT_TASKS:
        K[i].add(j)

    return TASKS, S, S_, rho, rho_, K

```

Figura 14: Caracterización de tareas

Fuente: Elaboración propia.

- **characterize_states:** Se definen los conjuntos vinculados a los estados, como los procesos que utilizan un estado como entrada, los que producen un nuevo estado y la capacidad asignada a cada uno. No obstante, se asume de manera predeterminada que los estados tienen capacidad ilimitada.

```

def characterize_states(STATES, ST_ARCS, TS_ARCS):
    # T[s] set of tasks receiving material from state s
    T = {s: set() for s in STATES}
    for (s,i) in ST_ARCS:
        T[s].add(i)

    # set of tasks producing material for state s
    T_ = {s: set() for s in STATES}
    for (i,s) in TS_ARCS:
        T_[s].add(i)

    # C[s] storage capacity for state s
    C = {s: STATES[s]['capacity'] for s in STATES}

    return T, T_, C

```

Figura 15: Caracterización de estados

Fuente: Elaboración propia.

create_maravelias_model: Esta función implementa el modelo matemático utilizando el framework de optimización Pyomo. En esta sección se describen los conjuntos, variables, función objetivo y restricciones que componen el modelo. A continuación, se detalla cómo se desarrolla con Pyomo:

1. Creación del modelo matemático: Para definir un modelo matemático se tienen dos opciones en Pyomo, el modelo concreto y abstracto, en el primer caso es aquel que posee sus parámetros definidos mientras que el abstracto no se posee dicha información y el framework otorga una instancia con datos aleatorios que se ajustan al problema definido. En el caso de la página web programada se utilizó el modelo concreto con el siguiente código:

```
model = ConcreteModel()
```

2. Definición variables: La manera de definir una variable con este framework resulta bastante sencillo, se debe especificar el tipo de variable, si es booleana o real. Además, se puede agregar los conjuntos asociados a cada variable como se muestra a continuación:

```

model.W = Var(TASKS, UNITS, GROUPS, TIME, domain=Boolean)
model.B = Var(TASKS, UNITS, GROUPS, TIME, domain=NonNegativeReals)
model.S = Var(STATES.keys(), GROUPS, TIME, domain=NonNegativeReals)

```

Figura 16: Definición de variables

Fuente: Elaboración propia.

3. La función objetivo representa el costo total asociado al almacenamiento durante el horizonte de planificación. Para esto, se utiliza la siguiente estructura:

```

Cost = {(s,t): STATES[s]['price']*(1+t) for s in STATES for t in TIME}
model.Value = Var(domain=NonNegativeReals)
model.valuec = Constraint(expr = model.Value == sum([Cost[s,t]*model.S[s,g,t] for s in STATES.keys() for g in GROUPS for t in TIME]))
model.obj = Objective(expr=model.Value, sense = minimize)

```

Figura 17: Función objetivo

Fuente: Elaboración propia.

Este planteamiento garantiza que los costos incrementen progresivamente con el tiempo, incentivando la eficiencia en la planificación.

4. Restricciones: Para la programación de las restricciones se creó una lista de éstas con el siguiente código:

```

model.cons = ConstraintList()

```

De esta forma se agregan las restricciones de forma iterativa con el comando:

```

model.cons.add(restriccion)

```

solve_maravelias_model: Esta función se utiliza para resolver el modelo matemático empleando el solucionador "CBC". Se configura con parámetros específicos, como límite de tiempo, un gap de optimalidad ajustable según la cantidad de entradas, y la búsqueda de soluciones iniciales factibles.

```

def solve_maravelias_model(model):
    solver = SolverFactory('cbc')
    solver.options['seconds'] = 60 * 60
    if cantidad_input > 70:
        solver.options['ratioGap'] = 10
    else:
        solver.options['ratioGap'] = 5
    solver.options['findInitial'] = True
    results = solver.solve(model)
    results.write()
    Condition = results.solver.termination_condition
    return Condition

```

Figura 18: Función solucionador modelo matemático

Fuente: Elaboración propia.

En la Figura 18 se puede observar cómo se implementan y ajustan parámetros relevantes en el solucionador CBC, como el límite de tiempo, el porcentaje de gap y la activación de la búsqueda de soluciones iniciales factibles. Estas configuraciones son fundamentales para mejorar la eficiencia del proceso de resolución. Desactivar algunas de estas opciones podría incrementar considerablemente el tiempo necesario para encontrar una solución viable.

characterize_units: Organiza datos sobre unidades, tareas, capacidades y tiempos de procesamiento. Identifica las unidades únicas, agrupa las tareas que cada unidad puede realizar, calcula las capacidades máxima y mínima por tarea, unidad y tiempo, y obtiene los tiempos de procesamiento de cada unidad.

```

def characterize_units(UNIT_TASKS, TIME):
    UNITS = set([j for (j,i) in UNIT_TASKS])

    # I[j] set of tasks performed with unit j
    I = {j: set() for j in UNITS}
    for (j,i) in UNIT_TASKS:
        I[j].add(i)

    # Bmax[(i,j,t)] maximum capacity of unit j for task i in the time t
    Bmax = {(i, j, t): UNIT_TASKS[(j, i)][ 'Bmax' ] for (j, i) in UNIT_TASKS for t in TIME}

    # Bmin[(i,j,t)] minimum capacity of unit j for task i in the time t
    Bmin = {(i, j, t): UNIT_TASKS[(j, i)][ 'Bmin' ] for (j, i) in UNIT_TASKS for t in TIME}

    pr = {j:UNIT_TASKS[(j,i)][ 'T. Proc' ] for (j,i) in UNIT_TASKS}

    return UNITS, I, Bmax, Bmin, pr

```

Figura 19: Función organización de datos

Fuente: Elaboración propia.

procesar_despalillado: Organiza y distribuye los kilos de uvas que se procesarán en máquinas de despalillado, respetando las capacidades máximas y trasladando los kilos excedentes al siguiente período de tiempo.

```

def procesar_despalillado(UNITS, GROUPS, TIME, model, R, Pt, Bmax):

    despalillado_data = []
    excess_kilos_handled = {}

    for j in UNITS:
        for g in GROUPS:
            for t in TIME:
                if model.W['Despalillado', j, g, t]() == 1:
                    kilos = R.get(('Racimo_uva', g, t), 0)
                    patentes = Pt.get((g, t), [])
                    bmax_actual = Bmax.get(('Despalillado', j, t), float('inf'))

```

Figura 20: Función distribución de kilos

Fuente: Elaboración propia.

obtener_cargas_entrantes: Recopila información sobre las cargas de uvas que llegan en cada instante de tiempo para distintos bloques o grupos. Su objetivo principal es identificar las cantidades de uvas que llegan a cada grupo en un momento determinado, utilizando la información de las patentes de los vehículos asociadas a esas cargas.

```
def obtener_cargas_entrantes(TIME, GROUPS, R, Pt):
    incoming_loads = []
    for t in TIME:
        for g in GROUPS:
            incoming_load = R.get(('Racimo_uva', g, t), 0)
            if incoming_load != 0:
                patentes = Pt.get((g, t), [])
                patentes_str = ", ".join(patentes)

                load_info = f"Para tiempo {t}: Llegan {incoming_load} Ton de bloque {g}.\nCon patentes: {patentes_str}\n\n"
                incoming_loads.append(load_info)

    return incoming_loads
```

Figura 21: Cantidad de uva por camión

Fuente: Elaboración propia.

generate_schedule: Organiza tareas programadas en un DataFrame indicando máquina, bloque, tiempo de inicio, duración, fin, cantidad procesada y lotes generados.

```
def generate_schedule(model, UNITS, I, GROUPS, TIME, pr):
    """Genera un DataFrame de programación basado en el modelo y los parámetros dados."""
    results = [
        {
            'Tarea': i,
            'Máquina': j,
            'Bloque': str(g),
            'Inicio': t,
            'Duración': pr[j],
            'Fin': t + pr[j],
            'Tamaño [Ton]': model.B[i, j, g, t](),
            'Lote de salida': round(
                model.B[i, j, g, t]() * (
                    0.96 if i == 'Despalillado' else
                    0.86 * 0.76 if i == 'Prensado' else
                    0.94 * 0.76 if i == 'Flotación' else
                    1 * 0.76
                ), 2
            )
        }
        for j in UNITS for i in I[j] for g in GROUPS for t in TIME if model.W[i, j, g, t]() > 0
    ]

    schedule = pd.DataFrame(results)
    return schedule
```

Figura 22: Función para generar DataFrame con la planificación

Fuente: Elaboración propia.

results_model: Organiza los resultados del modelo en una lista estructurada por tipo de tarea y máquina. Calcula tiempos de inicio, duración, fin y tamaño de lotes ajustados.

```

def results_model(model,I,UNITS,GROUPS,TIME,pr):
    RESULTS = []
    ORDERTASK = ['Despalillado','Prensado','Pre-flotación','Flotación','Fermentación']
    for m in ORDERTASK:
        results=[]
        if m == 'Fermentación' or m == 'Pre-flotación' or m=='Flotación':
            rho = 760/1000
        else:
            rho = 1
        result = [{'Machine': j,
                    'Block': g,
                    'Start': t,
                    'Duration': pr[j],
                    'Finish': t+pr[j],
                    'Batch': round(model.B[i,j,g,t]()*rho,2)}
                  for j in UNITS for i in I[j] for g in GROUPS for t in TIME if model.W[i,j,g,t]()>0]
        RESULTS.append(result)

    return RESULTS,ORDERTASK

```

Figura 23: Resultados ajustados

Fuente: Elaboración propia.

Unites: Toma una lista de tareas (ORDERTASK) y asigna una unidad de medida a cada tarea. Si la tarea es 'Fermentación', 'Pre-flotación' o 'Flotación', se asigna la unidad de medida '[kLt]'. Para todas las demás tareas, se asigna '[Ton]'. Al final, se añade una unidad de medida adicional '[Ton]' a la lista.

```

def unites(ORDERTASK):
    UnidadMedida = []
    for i in ORDERTASK:
        if i == 'Fermentación' or i == 'Pre-flotación' or i=='Flotación':
            UnidadMedida.append('\n[kLt]')
        else:
            UnidadMedida.append('\n[Ton]')
    UnidadMedida.append('\n[Ton]')
    return UnidadMedida

```

Figura 24: Función de transformación de unidades de medida

Fuente: Elaboración propia.

collection_truck: Esta función implementa un algoritmo para asignar 4 camiones a la recolección de escobajo y orujo, bajo la condición de que la cantidad acumulada de desechos supere las 30 toneladas. Los camiones pueden ser asignados nuevamente cada 8 horas, tiempo configurable según necesidad. Durante la ejecución, se registran datos clave de cada

asignación, como el camión utilizado, el bloque de tiempo, la duración, el tonelaje transportado y los periodos de disponibilidad.

visualize: Toma los resultados de una programación de tareas y genera un gráfico de Gantt para visualizar el cronograma de las tareas en diferentes máquinas. Convierte los resultados en un DataFrame, ajusta los parámetros según la tarea, define estilos y colores, y crea una figura con matplotlib. Dibuja líneas representando la duración de las tareas, ajusta las etiquetas del eje X, añade una leyenda y guarda el gráfico como un archivo PDF. Finalmente, muestra el gráfico y cierra la figura para liberar memoria.

generate_and_merge_pdfs: Esta función coordina la generación y fusión de varios archivos PDF. Primero, determina la variable HoraEje basada en la hora actual. Luego, incluye tres funciones internas encargadas de crear PDFs específicos: uno para las cargas entrantes y otro a partir de un DataFrame, además de una para guardar el DataFrame como imagen. Finalmente, fusiona todos los archivos PDF numerados del directorio de entrada en un único archivo, el cual se guarda con un nombre que incluye la fecha actual y HoraEje.

create_incoming_loads_pdf: Esta función genera un archivo PDF con una lista de cargas entrantes, colocando cada carga en una nueva línea. Para ello, utiliza la biblioteca FPDF y guarda el archivo en la ruta especificada. También confirma la generación exitosa mediante un mensaje en consola.

save_dataframe_as_image: Esta función guarda un DataFrame en formato de imagen utilizando matplotlib. Genera una tabla visual a partir del DataFrame, ajusta el tamaño de la fuente y las columnas, y guarda la imagen en la ruta especificada. Por último, confirma el guardado con un mensaje en consola.

create_dataframe_pdf: Esta función convierte una imagen de un DataFrame en un archivo PDF. Utiliza FPDF para crear el archivo en orientación horizontal e inserta la imagen ajustada al tamaño del PDF. El archivo final se guarda en la ruta especificada, confirmando el éxito del proceso, o muestra un error si la imagen no se encuentra.

6 Actualizaciones

6.1 Definir usuario: Configuración de Variables

- Abrir la terminal CMD como administrador.
- Configurar las variables de usuario y contraseña para la conexión al Cloud Foundry.
- Ejecutar los siguientes comandos:

```
setx cf_u_tesista "smartwinery@conchaytoro.cl" setx cf_p_tesista "Developer.2023.$$"
```

Figura 25: Configuración de Variables para Cloud Foundry

Fuente: Elaboración propia.

6.2 Añadir cambios

6.2.1 Actualización del Backend

Compilación de la Imagen Docker

- Abrir PowerShell dentro de Visual Studio, ubicado en la raíz del proyecto.
- Asegurarse de tener la aplicación Docker abierta.
 - Ejecutar el siguiente comando para compilar la imagen de Docker:

```
docker build -t dockerfile:djangoBackend .
```

Figura 26: Compilación de Imagen Docker - Backend

Fuente: Elaboración propia.

Subida de Imagen a Docker Hub

- Abrir la terminal CMD como administrador.
- Ingresar al sistema Docker mediante el comando de login.
- Ejecutar los siguientes comandos para etiquetar y subir la imagen a Docker Hub

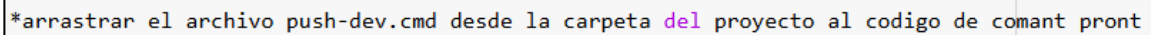
```
• docker login -u lucianoatm
• # Se solicitar la contraseña: gelatina1A
docker tag dockerfile:djangoBackend lucianoatm/capstonebackend:latest
docker push lucianoatm/capstonebackend:latest
```

Figura 27: Subida de imagen a Docker

Fuente: Elaboración propia

Despliegue en Cloud Foundry

- Dentro del proyecto en Visual Studio, abrir una terminal Command Prompt (cmd) ubicado en la raíz del proyecto.
- Arrastrar el archivo push-dev.cmd desde la carpeta del proyecto al código del comando prompt.



```
*arrastrar el archivo push-dev.cmd desde la carpeta del proyecto al código de comando prompt
```

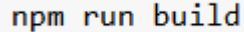
Figura 28: Despliegue en Cloud Foundry - Backend

Fuente: Elaboración propia.

6.2.2 Actualización del Frontend

Compilación del Frontend

- Abrir PowerShell dentro de Visual Studio, ubicado en la carpeta frontend.
- Ejecutar el siguiente comando para compilar el frontend antes de subirlo a Cloud Foundry:



```
npm run build
```

Figura 29: Compilación del Frontend

Fuente: Elaboración propia.

Despliegue en Cloud Foundry

- Desde el proyecto en Visual Studio, abrir una terminal Command Prompt (cmd) situado en la carpeta frontend.
- Llevar el archivo push-dev.cmd, ubicado en “Capstone\frontend”, desde la carpeta del proyecto y arrastrarlo al código del comando prompt.